Pattern A Operationalization: Cell-to-Adjacent Consultation as Standalone Architectural Specification in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher) **ORCID:** 0009-0004-8065-3235 **Date:** May 7, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the first of the three legitimate composition patterns named in the source paper's hybrid-systems-composition commitment — **Pattern A**, in which a CKS cell consults an adjacent AI component as input to its execution — as a standalone architectural specification, integrating the foundational commitments specifically operationalizing Pattern A's behavior: AI-as-substrate-mediator, path retraceability, the determinism contract, and substrate-as-source-of-truth.

Abstract

The CKS pattern's hybrid-systems-composition commitment names three legitimate positions an adjacent AI component can occupy relative to a CKS substrate (§4.5 of the source paper): cell-to-adjacent consultation (Pattern A), substrate-derived view (Pattern B), and separate concern (Pattern C). A separate note formalizes the three-patterns commitment as a whole; earlier composition-pair notes have formalized specific aspects of Pattern A — the mediator role's interaction with retraceability, with substrate authority, and across multiple adjacencies. None articulates Pattern A as a single architectural specification integrating all of the foundational commitments that operationalize its behavior. This note does. It states the four operational components Pattern A requires — unidirectional consultation flow, cell mediator under orchestration rules, consultation events retraceable through substrate provenance, substrate authority preserved through adjacent non-authoritativeness — distinguishes Pattern A from Pattern B, Pattern C, and the composition anti-patterns the source paper identifies, and provides an operational test sharpened by three named properties: Pattern-A-unidirectional-flow, Pattern-A-cell-mediator-rule-governance, and Pattern-A-adjacent-non-authoritative.

1. Why Pattern A operationalization needs to be formalized as standalone

The hybrid-systems-composition commitment names three legitimate positions for any adjacent AI component (§4.5): the adjacent component is consulted as input to a cell (Pattern A), is a derived view of substrate content (Pattern B), or handles a separate concern (Pattern C). What the architecture

forbids is a fourth position — and what makes a fourth position identifiable as a violation is the standalone specification of what each of the three legitimate positions requires.

Earlier composition-pair notes have formalized specific faces of Pattern A: the mediator-role × retraceability composition, the mediator-role × source-of-truth composition, and the mediator role across multiple adjacencies. Each covers one face. None covers Pattern A as a complete architectural specification.

That is what this note adds. Pattern A's operational behavior is not the sum of its faces taken separately; it is the integration of the foundational commitments that together specify how a cell consults an adjacent component without violating any of the architecture's other commitments. The integration is what makes Pattern A a position in the architecture rather than an implementation detail. Naming the operationalization as a standalone specification gives downstream implementations a single citation target, gives auditors a single specification to verify against, and gives critics a single architectural object to argue with.

This note opens the fifth and final tier of the composition-pair phase. Subsequent notes formalize Pattern B and Pattern C as standalone specifications; the closing note of the phase addresses the three-patterns coherence under which the specifications coexist.

2. The four operational components

Pattern A's architectural specification has four operational components. Each is grounded in a foundational commitment the source paper defends; none is a new commitment invented for the specification.

(a) Unidirectional consultation flow. Information flow under Pattern A is one-directional: the cell sends a consultation request to the adjacent component (cell → adjacent), the adjacent component returns a response (adjacent → cell), and the cell — having processed the response under its orchestration rule — writes its outputs back to the substrate (cell → substrate). The flow direction the architecture forbids is adjacent → substrate: the adjacent component does not write to the substrate, directly or via an automated propagation mechanism, regardless of how convenient the shortcut would be. This component grounds in §4.5.

(b) Cell mediator operates under orchestration rules. The consulting cell continues to operate as a substrate mediator for the duration of its execution, including the portion that consults the adjacent component. The consultation does not bypass orchestration-rule governance; the rule authorizes the cell's writes, and what those writes contain — including any content informed by the consultation — is bound by the rule. The adjacent component's response is input to the cell's reasoning, not authorization for the cell's writes. This component grounds in §4.4.

(c) Consultation events retraceable through substrate provenance. Every consultation event must be retraceable from substrate content alone (§3.1, §5). The cell-execution-id under which the consultation occurred, the orchestration rule that authorized the cell's execution, the adjacent

component consulted, and the cell's writes resulting from the consultation must all be addressable substrate content. The retraceable path crosses the cell–adjacent boundary cleanly: a reader querying substrate content can reconstruct that cell C, executing under rule R, consulted adjacent A and produced substrate content S as a result. The path does not stop at the consultation boundary; if it did, retraceability would fail at the moment the cell's reasoning crossed an opaque interface.

(d) Substrate authority preserved through adjacent non-authoritativeness. The adjacent component is consulted, not authoritative (§11.3). When the cell reads from both the adjacent and the substrate and the two disagree on a coordination question — what was decided, by whom, under what authority, with what rationale — the substrate wins by definition. The adjacent component does not become a substitute source of truth, regardless of how convenient or comprehensive its content may be. This component is the architectural protection against the gradual drift that turns a Pattern B derived view into a Pattern A consultation source treated as authoritative.

The four components are jointly necessary. A deployment that satisfies any three but violates the fourth has not implemented Pattern A; it has implemented something else, and the something else is identifiable as a composition anti-pattern.

3. What the operationalization adds beyond the general framework

The general framework names Pattern A as one of three legitimate positions and states what it requires at a high level. The operationalization makes three integration points architectural rather than implicit. *Unidirectionality* becomes a direct property of Pattern A rather than something inferable only by composing the substrate–cell-boundary commitment, the mediator-role commitment, and path retraceability. *Consultation-event-as-substrate-content* specifies what cleanly crossing the component boundary requires — provenance fields including cell-execution-id, with the consultation event itself recorded as addressable substrate content rather than as an externally logged runtime event. *Adjacent-non-authoritativeness as composition invariant* makes the corollary of substrate-as-source-of-truth architectural for Pattern A specifically: the adjacent component, when consulted under Pattern A, is non-authoritative across the consultation, and the substrate-substitute anti-pattern is identifiable at design time rather than at the moment a coordination question is answered from the wrong place.

4. What Pattern A is NOT

Stating what Pattern A is not keeps the standalone framing scoped.

Not Pattern B. Pattern B is the inverse flow direction (substrate → adjacent) and treats the adjacent as a derived projection of substrate content rather than as a separate component the cell reads from. The two patterns may coexist in a deployment; conflating them produces neither.

Not Pattern C. Pattern C is the separate-concern position: the adjacent component handles a concern that does not affect coordination state, with no architectural coupling to the substrate. Pattern A is

coupled by definition: the cell reads from the adjacent and writes to the substrate based on what it reads.

Not bidirectional consultation. A consultation in which the adjacent component subsequently writes to the substrate, directly or via automated propagation, is not Pattern A; it is the hidden-bidirectional-coupling anti-pattern. The fix is to bring the propagation under an orchestration rule — converting it into a separate Pattern A consultation, or into a Pattern B view rebuild.

Not adjacent-as-source-of-truth. A consultation in which the cell defers to the adjacent on coordination questions, treating the adjacent's content as the answer, is not Pattern A. It is the substrate-substitute anti-pattern. The cell may draw on the adjacent for additional context, source-document excerpts, or background; it cannot defer to it on what was decided, by whom, under what authority, or with what rationale.

Not consultation by an autonomous AI. A "consultation" performed by an autonomous agent rather than a substrate mediator is not Pattern A. Pattern A is unsalvageable when the cell's interior is not a mediator: there is no rule-governed write to receive the consultation result, and the consultation collapses into the agent's autonomous state-management.

5. Anti-patterns specifically violating Pattern A

Five failure modes recur in deployments that approximate Pattern A without satisfying its specification.

(a) Ungoverned writer. An adjacent component writes to the substrate outside cell mediation, with no orchestration-rule trace, no consultation provenance, and no authority constraints a cell would have applied. Violates AI-as-substrate-mediator, path retraceability, and the unidirectionality requirement.

(b) Hidden bidirectional coupling. The cell consults the adjacent, and the adjacent subsequently propagates state into the substrate via an automated process no orchestration rule governs. The substrate–cell boundary is silently bypassed, and path retraceability breaks because the propagation produces substrate content with no rule reference and no cell-execution-id. The fix is to bring each direction under an orchestration rule.

(c) Substrate substitute. A Pattern A consultation drifts into being authoritative — first because the adjacent's content is faster to query than the substrate's, then because participants reach for it first, then because new content is written through it. By the time the substrate is being reconstructed from the consultation source, the source of truth has migrated. Violates substrate-as-source-of-truth and the adjacent-non-authoritativeness component.

(d) Pattern A from autonomous agent. The "consulting cell" is not a mediator but an autonomous agent that holds substrate-relevant state outside the substrate and writes coordination state outside rule authorization. Pattern A is unsalvageable in this configuration.

(e) Pattern A bidirectional. A direct violation of the unidirectionality requirement, in which the consultation is described as "two-way" — the adjacent returns a response and is also expected to

update its own state from substrate writes the cell makes. The remedy is to decompose into a Pattern A consultation in one direction and a Pattern B derived view in the other, each under its own rule.

A deployment that exhibits any of (a)–(e) has not implemented Pattern A. Naming each failure precisely is what allows downstream remediation to address the actual violation rather than a symptom.

6. Operational decisions that follow from the specification

Four design-time decisions follow directly from the four components. *Cell consultation under Pattern A is documented with rule reference*: every cell consults under an orchestration rule that names the consultation, the adjacent component, and the conditions; the rule reference is recorded in the substrate alongside the cell's writes, so that an unreferenced consultation is, by construction, an ungoverned operation. *Consultation events are recorded with cell-execution-id*: the provenance carried by substrate content the cell writes after a consultation includes the cell-execution-id, the orchestration rule reference, and the adjacent component consulted. The consultation event is itself substrate content; the trace runs through the substrate, not through external logs. *Adjacent components are classified as non-authoritative at deployment time*: a deployment names each adjacent component and records its position, so that an ambiguous classification is a substrate-substitute risk identified at design time rather than at incident time. *Bidirectional needs are decomposed*: a deployment that needs information to flow in both directions decomposes the need into a Pattern A consultation in one direction and a Pattern B derived view in the other, each under its own rule — the architectural form of the substrate–cell-boundary commitment applied to the cell–adjacent boundary.

7. Operational test

A system implements Pattern A as a CKS-coherent composition if and only if all of the following are true at all times during the substrate's existence and across every cell-execution that consults an adjacent component:

1. **Pattern-A-unidirectional-flow.** Every consultation event consists of a cell → adjacent request, an adjacent → cell response, and (where the cell's rule authorizes it) a cell → substrate write that may draw on the response. The adjacent component does not write to the substrate, directly or via automated propagation, regardless of the rationale for doing so.

2. **Pattern-A-cell-mediator-rule-governance.** The consulting cell operates as a substrate mediator under an orchestration rule that authorizes the consultation. The adjacent component's response is input to the cell's reasoning, not authorization for the cell's writes; what the cell writes is bound by the rule and not by the response.

3. **Pattern-A-adjacent-non-authoritative.** The adjacent component is non-authoritative on coordination questions. When the adjacent and the substrate disagree on what was decided, by whom, under what authority, or with what rationale, the substrate wins by definition; the disagreement is not adjudicated.

4. Every consultation event is recorded in the substrate with the cell-execution-id, the orchestration rule reference, and the adjacent component consulted, such that a reader can reconstruct the consultation by reading substrate content alone.
5. Substrate content the cell writes following a consultation references the consultation event, allowing a downstream reader to trace back through the consultation to the rule and the cell-execution that produced it.

A system that fails any of (1)–(5) may be a useful system, and may compose adjacent components in some other useful way, but does not implement Pattern A in the CKS sense. The first three — Pattern-A-unidirectional-flow, Pattern-A-cell-mediator-rule-governance, Pattern-A-adjacent-non-authoritative — are the sharpening properties under which a deployment's Pattern A use is most reliably distinguished from the composition anti-patterns named in §5.

A one-sentence test: a deployment uses Pattern A if and only if every adjacent-component consultation produces, where it produces substrate state at all, exactly one cell-mediated, rule-authorized, retraceable write under which the adjacent component is consulted but not authoritative.

8. Why naming Pattern A operationalization as standalone matters

Pattern A is the most common of the three legitimate composition patterns in real CKS deployments; almost every hybrid deployment uses it for at least one adjacent component. Without a standalone specification, the pattern is implicit, and implicit patterns drift. A deployment that adopts Pattern A by inference from the general framework alone may satisfy the framework's high-level intent while violating unidirectionality, rule-governance, retraceability, or adjacent-non-authoritativeness invariants individually — and each violation looks like an engineering simplification rather than an architectural failure. Naming the four components and the three sharpening properties is what makes those individual violations identifiable at the moment a deployment decision is made.

This note opens the fifth and final tier of the composition-pair phase. Subsequent notes formalize Pattern B and Pattern C as standalone architectural specifications, and the closing note of the phase addresses the coherence under which the three patterns coexist in a single deployment without crowding each other into anti-patterns.

Subsequent work that adopts the CKS pattern, composes it with adjacent AI components, or argues against the three-patterns commitment should use Pattern A in the sense formalized here. Subsequent work that uses the term differently — to describe a bidirectional cell–adjacent relationship, a configuration in which the adjacent component is treated as authoritative, or a consultation by an autonomous agent — is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Pattern A Operationalization: Cell-to-Adjacent Consultation as Standalone Architectural Specification in the Coordination Knowledge Substrate Pattern.* May 7, 2026. ORCID: 0009-0004-8065-3235.